Operating System (BC03001011)

Important Questions with Model Answers — GTU BCA Sem 3 Internal Exam (30 Marks)

How to use: Unit 3 and Unit 4 carry the highest weightage (25% each), then Unit 2 and Unit 5 (20%), Unit 1 (10%). Write answers in points + a diagram where shown; for numericals always show the table/Gantt chart. 5-mark answers: ~8-10 points; 3-mark answers: 4-5 points.

UNIT 1: Introduction

Q1. What are the objectives and functions of an Operating System? [5 marks]

An **Operating System (OS)** is system software that acts as an interface between the user/applications and the computer hardware, and manages hardware and software resources.

Objectives:

- **Convenience** - makes the computer easier to use.
- **Efficiency** - allows computer resources (CPU, memory, I/O) to be used in an efficient manner.
- **Ability to evolve** - permits effective development, testing and introduction of new system functions without interfering with service.

Functions:

- **Process management** - creation, scheduling, termination of processes.
- **Memory management** - allocation/deallocation, protection, virtual memory.
- **File management** - creating, deleting, organizing and protecting files and directories.
- **I/O (device) management** - device drivers, buffering, spooling, disk scheduling.
- **Security and protection** - authentication and access control.
- **User interface** - CLI / GUI and system calls for programs.
- **Error detection and accounting** - detects hardware/software errors, keeps usage statistics.

```
User  <->  Application Programs
              |
       Operating System   (kernel + utilities)
              |
         Hardware (CPU, Memory, I/O devices)
```

Q2. Explain the evolution of Operating Systems. [5 marks]

Stage	Description	Key point
Serial processing	Programmer interacts directly with hardware using console lights/switches; machine time scheduled manually	Setup time wasted, no OS
Simple batch systems	Similar jobs batched together; a *monitor* (resident software) automatically loads next job	CPU idle during I/O; no interaction
Multiprogrammed batch	Several jobs kept in memory; CPU switches to another job when one waits for I/O	Better CPU utilisation; needs memory management and scheduling
Time-sharing systems	CPU time divided into small time slices (quantum); many users interact simultaneously	Fast response; multitasking
Distributed / Real-time / Modern	Multiple computers cooperating; deadline-based systems; multicore, mobile, cloud OS	Microkernel, threads, SMP

Q3. What are the OS design considerations for Multiprocessor and Multicore systems? [3 marks]

- **Simultaneous concurrent processes/threads** - kernel routines must be re-entrant so several processors can execute them at once.
- **Scheduling** - any processor may perform scheduling; synchronization conflicts must be avoided.
- **Synchronization** - locks/spinlocks, mutual exclusion on shared kernel data.
- **Memory management** - must deal with multiple processors sharing memory (cache coherence, paging).
- **Reliability and fault tolerance** - OS should degrade gracefully if one processor fails.

- Multicore additionally needs: parallelism within applications, virtual machine approach, and efficient thread scheduling across cores.

Q4. Define Fault Tolerance. Explain its fundamental concepts. [3 marks]

Fault tolerance is the ability of a system to continue operating correctly even when some hardware or software component fails.

- **Reliability** R(t) = probability system works correctly up to time t.
- **Availability** = fraction of time the system is available = MTTF / (MTTF + MTTR).
- **Mean Time To Failure (MTTF)** and **Mean Time To Repair (MTTR)**.
- Faults can be permanent, temporary (transient) or intermittent.
- OS mechanisms: process isolation, redundancy, checkpointing, retry, replication, graceful degradation.

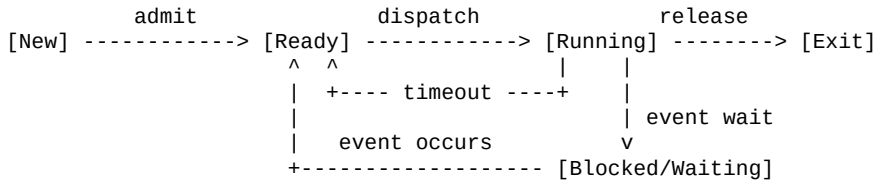
Q5. Write a short note on Traditional UNIX and Modern UNIX systems. [3 marks]

- **Traditional UNIX**: monolithic kernel with layers - hardware, kernel (system call interface, file subsystem, process control subsystem), libraries, user programs. Developed at Bell Labs (Ken Thompson, Dennis Ritchie).
- **Modern UNIX** (System V Release 4, BSD, Solaris, Linux): modular kernel, dynamically loadable modules, better threading, virtual memory, multiprocessor support, networking.
- Both are multiuser, multitasking, portable (written in C), hierarchical file system, shell as command interpreter.

UNIT 2: Process and Threads Management

Q6. What is a Process? Explain Process States with diagram (five-state model). [5 marks]

A **process** is a program in execution. It includes program code, program counter, registers, stack, heap and data section.



- **New** - process is being created.
- **Ready** - in memory, waiting for the CPU.
- **Running** - instructions are being executed.
- **Blocked/Waiting** - waiting for an event (I/O completion).
- **Exit/Terminated** - process has finished execution.
- Seven-state model adds **Ready/Suspend** and **Blocked/Suspend** states (processes swapped out to disk).

Transitions: Null-New (create), New-Ready (admit), Ready-Running (dispatch), Running-Ready (timeout/preempt), Running-Blocked (wait for event), Blocked-Ready (event occurs), Running-Exit (completion).

Q7. What is a Process Control Block (PCB)? Explain its contents. / Explain process description and control. [5 marks]

The **PCB** is a data structure maintained by the OS for every process; it is the 'identity card' of the process.

- **Process identifier (PID)**, parent PID, user ID.
- **Process state** - new/ready/running/waiting etc.
- **Program counter** - address of next instruction.
- **CPU registers** - accumulator, index, stack pointer, etc.
- **CPU scheduling info** - priority, scheduling queue pointers.
- **Memory management info** - base/limit registers, page tables.
- **Accounting info** - CPU time used, time limits.
- **I/O status info** - list of open files, allocated devices.

Process control includes: process creation (assign PID, allocate space, init PCB, link into ready queue), switching, and termination. **Process switch (context switch)**: save context of running process into its PCB, update state, move PCB to the proper queue, select another process, update its PCB, restore its context.

Q8. What is a Thread? Differentiate between Process and Thread. [5 marks]

A **thread** is the basic unit of CPU utilisation (a lightweight process). Threads of the same process share code, data and open files but have their own thread ID, program counter, register set and stack.

Process	Thread
Heavyweight, program in execution	Lightweight, unit of execution within a process
Has its own address space	Shares address space of its process
Context switch is slow	Context switch is fast
Inter-process communication needs OS support	Threads communicate easily via shared memory
Creation/termination takes more time	Creation/termination takes less time
Independent; failure of one does not affect others	One thread can affect other threads of same process

Q9. Explain Multithreading and types of threads (User-Level and Kernel-Level). Explain thread states. [5 marks]

Multithreading is the ability of an OS to support multiple concurrent paths of execution within a single process. Benefits: responsiveness, resource sharing, economy, scalability on multicore.

User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed by thread library in user space; kernel unaware	Managed directly by the kernel
Fast creation and switching (no kernel mode switch)	Slower, requires kernel mode switch
If one thread blocks, whole process blocks	Other threads continue if one blocks
Cannot use multiple processors simultaneously	Can run on multiple processors
Scheduling is application specific	Kernel can schedule threads individually

Multithreading models: many-to-one, one-to-one, many-to-many; also a combined approach (e.g. Solaris).

Thread states: Spawn, Ready, Running, Blocked, Finish. Thread operations: spawn, block, unblock, finish.

Q10. Explain Execution of the Operating System (kernel execution models). [3 marks]

- **Non-process kernel** - kernel executes outside any process, as a separate entity with its own privileged mode.
- **Execution within user processes** - OS software runs in the context of a user process (process image includes kernel stack); mode switch rather than process switch; used by UNIX.
- **Process-based OS** - OS implemented as a collection of system processes; modular design, good for multiprocessor.

UNIT 3: Concurrency and Deadlock

Q11. What is the Critical Section problem? State the requirements for Mutual Exclusion. Explain hardware support. [5 marks]

A **critical section** is a segment of code in which a process accesses shared resources; only one process should be in it at a time. **Race condition**: outcome depends on the order of execution of concurrent processes.

Requirements for mutual exclusion:

- **Mutual exclusion** - only one process in the critical section at a time.
- **Progress** - a process outside its critical section must not block others from entering.
- **Bounded waiting** - no process waits indefinitely (no starvation).
- No assumptions about relative speed or number of processors; no deadlock.

Hardware support:

- **Interrupt disabling** - works only on uni-processor, degrades efficiency.
- **Test-and-Set instruction** - atomically reads and sets a lock variable.
- **Exchange (swap) instruction** - atomically swaps a register and a memory location.

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = TRUE;
    return rv;
```

```
do {
    while (TestAndSet(&lock)) ;   // busy wait
    /* critical section */
    lock = FALSE;
```

```
}                                           /* remainder section */
                                          } while (TRUE);
```

Advantages: simple, applies to any number of processes. **Disadvantages:** busy waiting, starvation and deadlock possible.

Q12. What is a Semaphore? Explain counting and binary semaphores. Solve Producer-Consumer problem using semaphores. [5 marks]

A **semaphore** is an integer variable accessed only through two atomic operations: **wait(P)** and **signal(V)**.

```
wait(S):   while (S <= 0) ;  S--;        signal(S):  S++;
```

- **Counting semaphore** - value ranges over an unrestricted domain; controls access to a resource with several instances.
- **Binary semaphore (mutex)** - value is 0 or 1; used for mutual exclusion.

Producer-Consumer (bounded buffer, size n): semaphores *mutex*=1, *empty*=n, *full*=0.

```
Producer:                    Consumer:
 do {                         do {
   produce item;                wait(full);
   wait(empty);                 wait(mutex);
   wait(mutex);                 remove item from buffer;
   add item to buffer;          signal(mutex);
   signal(mutex);               signal(empty);
   signal(full);                consume item;
 } while(TRUE);               } while(TRUE);
```

Q13. Explain Monitors. How are they different from semaphores? [3 marks]

A **monitor** is a high-level synchronization construct: a software module consisting of shared variables, procedures that operate on them, and initialization code. Only one process can be active inside a monitor at a time (mutual exclusion is automatic). Synchronization inside a monitor uses **condition variables** with operations **cwait(c)** and **csignal(c)**.

- Semaphores are low-level; programmer must place wait/signal correctly (errors cause deadlock). Monitors are language-level and safer.
- In monitors, mutual exclusion is implicit; with semaphores it is explicit.
- Condition variables have no memory (signal with no waiter is lost), semaphores keep a count.

Q14. Explain Message Passing (send / receive). [3 marks]

- Provides both **synchronization** and **communication** using primitives **send(destination, message)** and **receive(source, message)**; works on shared-memory and distributed systems.
- **Synchronization:** blocking send/blocking receive (rendezvous), non-blocking send/blocking receive, non-blocking both.
- **Addressing:** direct (explicit process names) or indirect (mailboxes / ports).
- **Message format:** header (source, destination, length, type) + body; queuing discipline FIFO or priority.
- Mutual exclusion can be achieved using a mailbox containing a single token message.

Q15. Explain the Readers-Writers problem and its solution using semaphores. [5 marks]

A shared data set is accessed by **readers** (only read) and **writers** (read and write). Conditions: any number of readers may read simultaneously; only one writer may write at a time and no reader may read while a writer writes.

First readers-writers solution (reader priority): shared: *readcount*=0, semaphores *mutex*=1, *wrt*=1.

```
Writer:                      Reader:
 do {                         do {
   wait(wrt);                   wait(mutex);
   /* writing */                readcount++;
   signal(wrt);                 if (readcount == 1) wait(wrt);
 } while(TRUE);                 signal(mutex);
                                /* reading */
                                wait(mutex);
                                readcount--;
                                if (readcount == 0) signal(wrt);
                                signal(mutex);
                              } while(TRUE);
```

Drawback: writers may starve. In the writer-priority variant, readers may starve.

Q16. What is Deadlock? Explain the four necessary conditions for deadlock. [5 marks]

Deadlock is a situation in which a set of processes is blocked permanently because each process holds a resource and waits for another resource held by some other process in the set.

- **Mutual exclusion** - at least one resource is non-sharable.
- **Hold and wait** - a process holding resources is waiting for additional ones.
- **No preemption** - resources cannot be forcibly taken away; released only voluntarily.
- **Circular wait** - a closed chain of processes exists where each waits for a resource held by the next.

All four conditions must hold simultaneously. Example: P1 holds R1 and requests R2, while P2 holds R2 and requests R1. Deadlock can be represented using a **Resource Allocation Graph** (a cycle indicates deadlock when each resource has a single instance).

Q17. Explain Deadlock Prevention. [5 marks]

Prevention ensures that at least one of the four necessary conditions can never hold.

Condition	How to prevent	Drawback
Mutual exclusion	Make resources sharable (e.g. read-only files)	Not possible for printers, etc.
Hold and wait	Request all resources at once before execution, or release held resources before requesting new	Low resource utilisation, starvation
No preemption	If a request cannot be granted, process releases all held resources; or OS preempts resources of waiting process	Only for resources whose state can be saved (CPU, memory)
Circular wait	Impose a linear ordering of resource types; processes request in increasing order only	Inconvenient ordering, reduced efficiency

Q18. Explain Deadlock Avoidance - Banker's Algorithm with example. [5 marks]

Avoidance requires advance information about maximum resource needs; the OS grants a request only if the resulting state is **safe**. A **safe state** is one in which there exists a sequence of processes (safe sequence) so that every process can finish. Unsafe state may lead to deadlock.

Data structures: Available[m], Max[n][m], Allocation[n][m], Need[n][m] = Max - Allocation.

Safety algorithm: (1) Work = Available, Finish[i] = false. (2) Find i with Finish[i]=false and Need[i] <= Work. (3) Work = Work + Allocation[i]; Finish[i]=true; repeat. (4) If all Finish[i] true, system is safe. **Resource-request algorithm:** if Request <= Need and Request <= Available, pretend to allocate; if the new state is safe, grant; else process waits.

Example: 5 processes, resources A,B,C; Available = (3,3,2).

Process	Allocation A B C	Max A B C	Need A B C
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Work=(3,3,2): P1 (need 1,2,2) runs -> Work=(5,3,2); P3 -> (7,4,3); P4 -> (7,4,5); P0 -> (7,5,5); P2 -> (10,5,7). **Safe sequence: <P1, P3, P4, P0, P2>**, so the system is in a safe state.

Q19. Explain Deadlock Detection and Recovery. [3 marks]

- **Detection:** allow deadlock to occur, then detect it. For single-instance resources use a **wait-for graph** (cycle = deadlock). For multiple instances use an algorithm similar to Banker's (Allocation, Request, Available vectors); processes that cannot finish are deadlocked.
- The detection algorithm is invoked periodically or when CPU utilisation drops.
- **Recovery by process termination:** abort all deadlocked processes, or abort one at a time until the cycle is broken (choose by priority, run time, resources held).
- **Recovery by resource preemption:** select a victim, rollback the process to a safe state, avoid starvation by limiting number of rollbacks.

Q20. Explain the Dining Philosophers problem and its solution. [5 marks]

Five philosophers sit around a table; there is one chopstick between each pair (5 total). A philosopher alternately thinks and eats; to eat, he needs both the left and right chopsticks. If all pick up their left chopstick simultaneously, each waits forever for the right one -> **deadlock** (and starvation is possible).

```
semaphore chopstick[5] = {1,1,1,1,1};
Philosopher i:
  do {
     wait(chopstick[i]);
     wait(chopstick[(i+1) % 5]);
     /* eat */
     signal(chopstick[i]);
     signal(chopstick[(i+1) % 5]);
     /* think */
  } while(TRUE);          // this basic version can deadlock
```

Deadlock-free remedies:

- Allow at most 4 philosophers at the table at once (extra semaphore room=4).
- Pick up both chopsticks only if both are available (inside a critical section / monitor).
- Asymmetric solution: odd philosophers pick left first, even philosophers pick right first.
- Monitor solution with states THINKING, HUNGRY, EATING and condition variable per philosopher.

UNIT 4: Memory Management and Scheduling

Q21. What are the requirements of Memory Management? [3 marks]

- **Relocation** - programs can be loaded at different memory locations; logical addresses translated to physical addresses.
- **Protection** - one process must not access another's memory; checked at run time by hardware.
- **Sharing** - controlled sharing of common code/data (e.g. shared libraries).
- **Logical organization** - programs are organized into modules (segments).
- **Physical organization** - management of flow between main memory and secondary storage.

Q22. Explain Memory Partitioning (fixed and dynamic) and fragmentation. [5 marks]

Fixed partitioning	Dynamic partitioning
Memory divided into fixed-size partitions (equal or unequal) at system generation	Partitions created dynamically, exactly the size a process needs
Causes **internal fragmentation** (unused space inside partition)	Causes **external fragmentation** (scattered free holes)
Degree of multiprogramming limited by number of partitions	No limit by partition count; solved by **compaction**
Simple, little OS overhead	Complex, compaction is time consuming

Placement algorithms (dynamic): First-fit (first hole big enough - fastest), **Best-fit** (smallest hole big enough - leaves tiny holes), **Worst-fit** (largest hole), **Next-fit**. **Buddy system** is a compromise between fixed and dynamic partitioning.

Q23. Explain Paging with address translation. What is page table structure? [5 marks]

Paging divides physical memory into fixed-size blocks called **frames** and logical memory into blocks of the same size called **pages**. Pages are loaded into any available frames; there is no external fragmentation (only small internal fragmentation in the last page).

Logical address = (page number p, offset d). Page table maps p to frame number f. **Physical address** = f x page size + d.

```
CPU --> [ p | d ] ---> Page Table [ p -> f ] ---> [ f | d ] --> Physical Memory
```

Example: page size 1 KB, page 2 is in frame 5; logical address (2, 100) -> physical = 5 x 1024 + 100 = 5220.

Page table structures: single-level, **multilevel (hierarchical)** page table (page table itself is paged), **inverted page table** (one entry per frame, hashed), each entry has frame number + present, modified, protection bits.

Q24. Explain Segmentation. Differentiate between Paging and Segmentation. [5 marks]

Segmentation divides a program into variable-length logical units (main, functions, stack, data). Logical address = (segment number s, offset d). The **segment table** stores base and limit for each segment; if d < limit, physical address = base + d, else trap (addressing error).

Paging	Segmentation
Fixed-size pages	Variable-size segments
Invisible to the programmer	Visible to programmer (logical view)
No external fragmentation, has internal fragmentation	Has external fragmentation, no internal fragmentation
Page table: frame number	Segment table: base + limit
Address = page no + offset	Address = segment no + offset
Does not support sharing/protection by logical unit easily	Easier sharing and protection of modules

Q25. What is Virtual Memory? Explain demand paging, page fault and TLB. [5 marks]

Virtual memory is a technique that allows execution of processes that are not completely in main memory; logical address space can be much larger than physical memory, using secondary storage as an extension. Based on locality of reference. Terms: *virtual address, virtual address space, address space, real address*.

Demand paging: a page is brought into memory only when it is referenced. A valid/invalid bit in the page table tells if the page is in memory.

Page-fault handling steps:

- Reference to a page whose valid bit is invalid -> trap to OS (page fault).
- OS checks if the reference is legal; if illegal, terminate process.
- Find a free frame (or select a victim using a page replacement algorithm).
- Schedule disk read to load the page into the frame.
- Update the page table (valid bit = 1) and restart the instruction.

TLB (Translation Lookaside Buffer): a small, fast associative cache of recently used page-table entries. On a TLB hit the frame number is obtained directly (one memory access); on a TLB miss the page table in memory is consulted (extra memory access) and the TLB is updated. **Effective Access Time** = h x (t_{tlb} + t_{mem}) + (1-h) x (t_{tlb} + 2 t_{mem}), h = hit ratio.

Q26. Explain Fetch policy, Placement policy and Replacement policy. [3 marks]

- **Fetch policy** - when a page should be brought into memory: **demand paging** (only on reference) or **prepaging** (bring several pages in advance).
- **Placement policy** - where in real memory a process piece is to reside; in pure paging/segmentation any frame is fine; in segmentation, best-fit/first-fit used.
- **Replacement policy** - which page to replace when no free frame exists: global vs local replacement, frame locking, algorithms like FIFO, LRU, Optimal, Clock.

Q27. Explain Page Replacement Algorithms (FIFO, LRU, Optimal). Solve: reference string 7,0,1,2,0,3,0,4,2,3,0,3,2 with 3 frames. [5 marks]

- **FIFO** - replace the oldest page in memory. Simple; may suffer **Belady's anomaly** (more frames, more faults).
- **LRU** - replace the page not used for the longest time; good performance, needs hardware support (counter/stack).
- **Optimal (OPT)** - replace the page that will not be used for the longest time in future; lowest faults but not implementable (used as benchmark).
- **Clock** - circular buffer with use bit (approximates LRU).

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
FIFO	F	F	F	F	H	F	F	F	F	F	F	H	H
LRU	F	F	F	F	H	F	H	F	F	F	F	H	H
OPT	F	F	F	F	H	F	H	F	H	H	F	H	H

(F = page fault, H = hit)

Algorithm	Page faults	Hits	Hit ratio
FIFO	10	3	3/13
LRU	9	4	4/13
Optimal	7	6	6/13

Frame contents (FIFO): [7] [7,0] [7,0,1] [2,0,1] hit [2,3,1] [2,3,0] [4,3,0] [4,2,0] [4,2,3] [0,2,3] hit hit. Optimal gives the minimum 7 faults.

Q28. Explain the types of Processor Scheduling. [3 marks]

- **Long-term (job) scheduling** - decides which jobs are admitted to the system; controls the degree of multiprogramming.
- **Medium-term scheduling** - swapping; part of the swapping function (suspends/resumes processes).
- **Short-term (CPU) scheduling / dispatcher** - selects which ready process executes next; runs very frequently (clock interrupts, I/O interrupts, system calls).
- **I/O scheduling** - decides which process's I/O request is handled by an available device.

Criteria: CPU utilisation, throughput, turnaround time, waiting time, response time, fairness. **Formulas:** TAT = Completion - Arrival; WT = TAT - Burst.

Q29. Explain CPU Scheduling Algorithms (FCFS, SJF, Round Robin, Priority). Solve the numerical. [5 marks]

- **FCFS** - non-preemptive, processes served in arrival order; simple but convoy effect, long waiting time.
- **SJF** - process with shortest burst time next; optimal average waiting time; may starve long jobs; preemptive version = **SRTF**.
- **Round Robin** - each process gets a time quantum, then goes to the back of the ready queue; preemptive, good for time-sharing; performance depends on quantum size.
- **Priority** - highest priority first; starvation solved by **aging**.

Numerical: P1 (AT 0, BT 5), P2 (AT 1, BT 3), P3 (AT 2, BT 8), P4 (AT 3, BT 6).

FCFS Gantt:

```
| P1 | P2 | P3 |   P4   |
0    5    8    16       22
```

SJF (non-preemptive) Gantt:

```
| P1 | P2 |  P4  |   P3   |
0    5    8     14       22
```

Round Robin (q = 3) Gantt:

```
| P1 | P2 | P3 | P4 | P1 | P3 | P4 | P3 |
0    3    6    9    12   14   17   20   22
```

Algorithm	Waiting time (P1,P2,P3,P4)	Avg WT	Turnaround (P1,P2,P3,P4)	Avg TAT
FCFS	0, 4, 6, 13	5.75	5, 7, 14, 19	11.25
SJF (NP)	0, 4, 12, 5	5.25	5, 7, 20, 11	10.75
RR (q=3)	9, 2, 12, 11	8.5	14, 5, 20, 17	14

Conclusion: SJF gives the minimum average waiting time here; RR gives better response time.

UNIT 5: I/O and File Management

Q30. Explain I/O Devices and Organization of the I/O Function. [3 marks]

- **I/O device categories:** human readable (printer, keyboard, display), machine readable (disks, sensors), communication (modems, NIC).
- **Organization of I/O function (evolution):**
- 1. **Programmed I/O** - CPU issues command and busy-waits for completion.
- 2. **Interrupt-driven I/O** - CPU continues other work; device interrupts on completion.
- 3. **Direct Memory Access (DMA)** - DMA controller transfers a block directly between device and memory; CPU is interrupted only once at the end.

- 4. **I/O channel / I/O processor** - separate processor executes I/O program.

Q31. What is I/O Buffering? Explain types of buffering. [5 marks]

Buffering is the use of a memory area to hold data temporarily during transfer between device and process; it smooths the speed mismatch and lets a process be swapped out during I/O.

- **Single buffer** - OS assigns one buffer in system memory; while one block is being processed the next is read ahead. Time per block = max(C, T) + M.
- **Double buffer** - two buffers; one is filled while the other is emptied; time per block = max(C, T).
- **Circular buffer** - more than two buffers used in a ring; suits bursty I/O (producer-consumer).
- **No buffering** - process waits for each I/O directly.

(T = I/O transfer time, C = computation time, M = time to move data between buffers.) Block-oriented devices (disks) and stream-oriented devices (terminals, printers) both use buffering.

Q32. Explain Disk Scheduling algorithms. Solve: queue 98,183,37,122,14,124,65,67 with head at 53. [5 marks]

Disk access time = seek time + rotational latency + transfer time. Disk scheduling mainly reduces seek time.

- **FCFS** - serve in arrival order; fair but large head movement.
- **SSTF** - nearest cylinder first; may cause starvation.
- **SCAN (elevator)** - head moves in one direction serving requests to the end, then reverses.
- **C-SCAN** - serves in one direction only; on reaching the end returns to start without serving; uniform wait time.
- **LOOK / C-LOOK** - like SCAN/C-SCAN but go only as far as the last request.

Algorithm	Head movement sequence	Total head movement
FCFS	53-98-183-37-122-14-124-65-67	45+85+146+85+108+110+59+2 = **640**
SSTF	53-65-67-37-14-98-122-124-183	12+2+30+23+84+24+2+59 = **236**
SCAN (towards 0)	53-37-14-0-65-67-98-122-124-183	16+23+14+65+2+31+24+2+59 = **236**

Q33. Explain RAID and its levels. [5 marks]

RAID (Redundant Array of Independent Disks) is a set of physical disk drives viewed by the OS as a single logical drive, with data distributed across drives (striping) and redundant parity/mirror information for recovery. It improves performance and reliability.

Level	Description	Features
RAID 0	Striping, no redundancy	Best performance, no fault tolerance
RAID 1	Mirroring (every disk duplicated)	High reliability, 50% capacity, costly
RAID 2	Bit-level striping with Hamming code	Rarely used
RAID 3	Byte-level striping, single dedicated parity disk	Good for large sequential transfers
RAID 4	Block-level striping, dedicated parity disk	Parity disk is a bottleneck
RAID 5	Block-level striping, parity distributed across all disks	Good balance; survives 1 disk failure
RAID 6	Block-level striping, two distributed parity blocks	Survives 2 disk failures

Q34. Explain File Organization and Access methods. [3 marks]

- **File organization:** pile (unordered), sequential file, indexed sequential file, indexed file, direct (hashed) file.
- **Sequential access** - records processed in order (read next, write next); tapes, editors.
- **Direct (random) access** - read/write any block directly by block number; databases.
- **Indexed access** - an index holds pointers to blocks; search the index first, then access data directly (ISAM).

Q35. Explain File Directories - structures. [5 marks]

A **directory** is a collection of nodes containing information about files (name, type, location, size, protection, time stamps). Operations: search, create, delete, list, rename, traverse.

- **Single-level directory** - all files in one directory; naming problem, no grouping.

- **Two-level directory** - separate User File Directory (UFD) for each user under a Master File Directory (MFD); solves name conflicts, no grouping.
- **Tree-structured directory** - hierarchy with root, absolute and relative path names; most common (UNIX, Windows).
- **Acyclic-graph directory** - allows shared subdirectories/files using links; issues with deletion and dangling pointers.
- **General graph directory** - cycles allowed; needs garbage collection.

Q36. Explain File Allocation Methods (Contiguous, Linked, Indexed). [5 marks]

Method	Description	Advantages	Disadvantages
Contiguous	Each file occupies a set of contiguous blocks; directory has start block and length	Simple, fast sequential and direct access, minimal seek	External fragmentation, file size must be known, hard to grow
Linked	Each block contains pointer to next block; directory has first (and last) block	No external fragmentation, files can grow easily	No efficient direct access, pointer overhead, reliability (lost pointer); FAT improves this
Indexed	A separate index block holds pointers to all blocks of file	Supports direct access, no external fragmentation	Index block overhead, large files need multilevel/linked index (UNIX inode)

```
Contiguous:  [ file A: blocks 5,6,7,8 ]
Linked:      block 9 -> block 16 -> block 1 -> block 25 -> end
Indexed:     index block [ 9 | 16 | 1 | 25 ]  each points to a data block
```

Q37. Explain Free Space Management techniques. [5 marks]

- **Bit vector (bitmap)** - one bit per block (1 = free, 0 = allocated). Simple, easy to find contiguous blocks; bitmap must be kept in memory, takes space.
- **Linked list** - free blocks linked together, pointer to first free block; no waste of space but traversal is slow.
- **Grouping** - first free block stores addresses of n free blocks; last of those points to the next group.
- **Counting** - store address of first free block and number (n) of contiguous free blocks following it.
- **Space maps / free list in tables** - used in advanced file systems like ZFS.

File system architecture (layers): Application programs -> Logical file system -> File-organization module -> Basic file system -> I/O control (device drivers, interrupt handlers) -> Devices.

QUICK REVISION: One-Line Definitions (for 1-2 mark questions)

Term	Definition
Kernel	Core part of the OS that always resides in memory and manages CPU, memory and devices
System call	Interface through which a user program requests a service from the OS kernel
Context switch	Saving state of the current process and loading state of the next one
Thrashing	Excessive paging activity; CPU spends more time swapping pages than executing
Belady's anomaly	Page faults increase when number of frames increases (in FIFO)
Starvation	A process waits indefinitely because others always get preference
Race condition	Result depends on the order in which concurrent processes access shared data
Spooling	Simultaneous Peripheral Operations On-Line - buffering output on disk (e.g. print queue)
Internal fragmentation	Wasted space inside an allocated block/partition
External fragmentation	Free memory exists but is not contiguous
Dispatcher	Module that gives CPU control to the process selected by short-term scheduler
Convoy effect	Short processes wait behind a long process in FCFS
Locality of reference	Programs tend to reference a small set of pages repeatedly during a time period

All the best for your exam!